

TOKO — Operation Manual

คู่มือปฏิบัติงานระบบ TOKO (Server / Scheduler / Earnings Flow)

TOKO DevOps

2026-05-06

สารบัญ

1. ภาพรวมระบบ (Overview)	2
2. โครงสร้างไฟล์สำคัญบนเครื่อง prod	3
3. Scheduler & Cron	4
3.1 ภาพรวม	4
3.2 Tasks ที่กำหนดไว้	4
3.3 Cron entry มาตรฐาน	4
3.4 ใครเป็นคน install cron นี้	4
4. Flow การโยกเงิน (Earnings Release Flow)	5
4.1 Concept	5
4.2 ขั้นตอน end-to-end	5
4.3 ตารางที่เกี่ยวข้อง	6
4.4 กฎการคำนวณ <code>release_at</code>	6
5. Operation Procedures (วิธีปฏิบัติ)	7
5.1 ตรวจสอบสถานะ scheduler	7
5.2 รับโยกเงินด้วยมือ (catch-up หลัง cron พัง)	7
5.3 ตรวจสอบรายได้ฐานใน DB	7
5.4 ดู <code>log error_log()</code> ของ PHP	7
6. Troubleshooting	8
6.1 รายได้ค้างไม่ปล่อยหลายวัน	8
6.2 Cron ไม่อ่าน DB credentials	8
6.3 ยอดถอน / <code>next_day</code> ไม่ตรง	8
7. Deploy Checklist	9
8. Reference	10
9. Notification Resend & Auto-Reassign	10

1. ภาพรวมระบบ (Overview)

ระบบ TOKO ประกอบด้วย 3 ส่วนหลัก

ส่วน	เทคโนโลยี	Repo
Backend API	Laravel 7 + PHP 7.4 (FPM)	<code>toko-server-v2</code>
Merchant App	Flutter (iOS/Android)	<code>toko-merchant-v2</code>
SuperApp (ลูกค้า)	Flutter	<code>toko-app-v2</code> (branch <code>toko_superapp_5</code>)

Hosting

- AWS Elastic Beanstalk (Amazon Linux 2, PHP 7.4 FPM)
- RDS MySQL 5.7 — `tokoprod.ceagkqnxap2h.ap-southeast-1.rds.amazonaws.com:12121` / db `tokodb`
- S3 bucket `tokobucketimg` (รูป/เอกสาร)
- CodeCommit -> CodePipeline -> Beanstalk (deploy ผ่าน `git push codecommit main`)

SSH เข้าเครื่อง prod

```
ssh ssm.toko.service
```

```
# ใช้ AWS SSM agent (ไม่ต้องเปิด port 22)
```

2. โครงสร้างไฟล์สำคัญบนเครื่อง prod

Path	คำอธิบาย
/var/app/current/	โค้ด Laravel ปัจจุบัน
/var/app/current/storage/logs/	log ของ Laravel + scheduler tasks
/opt/elasticbeanstalk/ deployment/env	env file ของ EB (mode 0400 root:root)
/etc/cron.d/laravel-scheduler	cron entry สำหรับรัน schedule:run ทุกนาที
/var/log/laravel-scheduler.log	output ของ php artisan schedule:run
/var/log/php-fpm/www-error.log	output ของ error_log() ใน PHP code

3. Scheduler & Cron

3.1 ภาพรวม

Scheduler รันผ่าน cron ทุกนาที โดยเรียก `php artisan schedule:run` ซึ่งจะตัดสินใจว่ามี task ไหนถึงเวลาทำงานบ้าง (กำหนดใน `app/Console/Kernel.php`)

3.2 Tasks ที่กำหนดไว้

Command	ความถี่	วัตถุประสงค์	Log file
<code>wallet:process-jobs</code>	ทุก 1 นาที	ประมวลผล wallet inquiry queue	<code>storage/logs/wallet-jobs.log</code>
<code>earning:release-pending</code>	ทุก 1 นาที	โยกรายได้ pending -> released เมื่อถึงเวลา	<code>storage/logs/earning-release.log</code>
<code>booking:auto-cancel-expired</code>	ทุก 1 นาที	ยกเลิกการจองโต๊ะที่หมดเวลารอ	<code>storage/logs/booking-auto-cancel.log</code>

ทุก task ใช้ `withoutOverlapping()` กันรับซ้อน

3.3 Cron entry มาตรฐาน

```
# /etc/cron.d/laravel-scheduler
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
* * * * * root bash -lc 'set -a; . /opt/elasticbeanstalk/deployment/env; set +a; \
  cd /var/app/current && sudo -E -u webapp php artisan schedule:run' \
  >> /var/log/laravel-scheduler.log 2>&1
```

สำคัญ: ต้องรัน cron เป็น root เพราะ `/opt/elasticbeanstalk/deployment/env` อ่านได้เฉพาะ root จากนั้นค่อย `sudo -E -u webapp` เพื่อให้ artisan รันด้วย user ที่เป็นเจ้าของไฟล์ใน `storage/` ถ้ารันเป็น `webapp` ตรงๆ -> `source env` ไม่ได้ -> DB fallback เป็น `localhost` -> ทุก task fail ด้วย `SQLSTATE[HY000] [2002] Connection refused`

3.4 ใครเป็นคน install cron นี้

ทุกครั้งที่ EB deploy เสร็จ จะรัน hook:

```
.platform/hooks/postdeploy/03_install_scheduler_cron.sh
```

ซึ่งจะ regenerate ไฟล์ `/etc/cron.d/laravel-scheduler` และ reload `crond` อัตโนมัติ -> instance ใหม่จาก auto-scaling จะได้ cron ที่ถูกต้องตั้งแต่บูต

4. Flow การโยกเงิน (Earnings Release Flow)

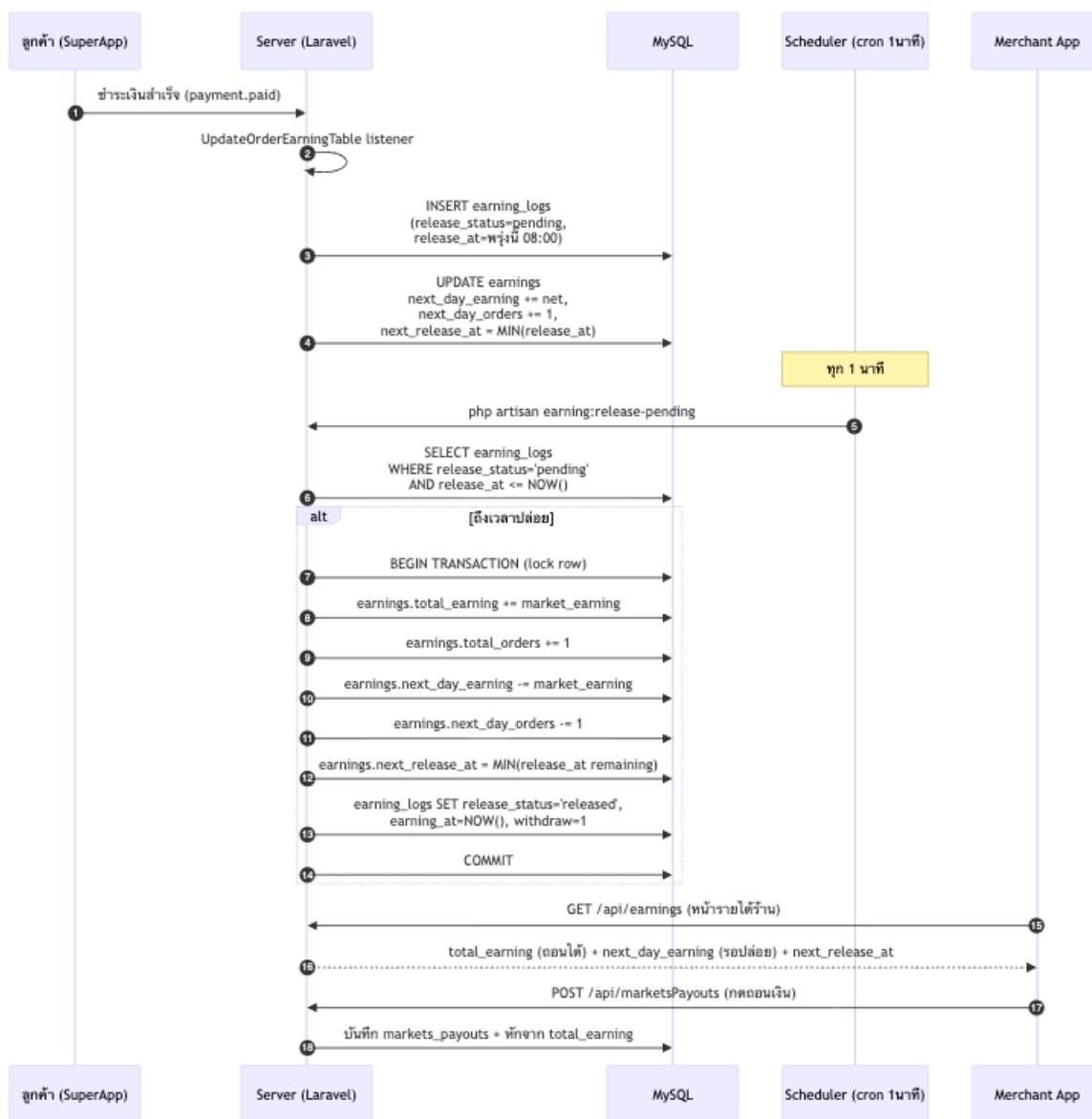
4.1 Concept

รายได้ของร้านค้าแบ่งเป็น 2 ก้อน

ก้อน	ฟิลด์ใน earnings	ความหมาย
รายได้ก่อนออกได้	total_earning	ใช้กดถอนได้ทันที
รายได้รอปล่อย (T+1)	next_day_earning	ออเดอร์ใหม่ที่ยังไม่ถึงเวลาปล่อย

แต่ละออเดอร์จะถูกบันทึก 1 record ในตาราง `earning_logs` ด้วย `release_status = pending` และ `release_at =` วันถัดไป 08:00 (Asia/Bangkok)

4.2 ขั้นตอน end-to-end



4.3 ตารางที่เกี่ยวข้อง

earning_logs

คอลัมน์	ประเภท	ความหมาย
market_id	int	ร้านค้าเจ้าของรายได้
order_id	string	reference ไปยังออเดอร์
order_paid_at	datetime	เวลาที่ลูกค้าจ่ายเงิน
release_at	datetime	กำหนดเวลาที่จะปล่อยเงินก่อนนี้
release_status	enum	pending / released
earning_at	datetime	เวลาที่ปล่อยเงินจริง (cron set)
order_amount	decimal	ยอดเดิมของออเดอร์
commission_rate	decimal	% commission ที่หัก
commission_amount	decimal	จำนวนเงินที่หัก
market_earning	decimal	ยอด net ที่ร้านได้รับ
withdraw	tinyint	1 = นับเข้า total_earning แล้ว

earnings (สรุป per-market)

คอลัมน์	ความหมาย
total_earning	ยอดสะสม “ก่อนได้”
total_orders	จำนวนออเดอร์ที่ปล่อยแล้ว
next_day_earning	ยอดรอบปล่อย
next_day_orders	จำนวนออเดอร์รอบปล่อย
next_release_at	เวลาที่ใกล้ที่สุดที่จะมีการปล่อยรอบถัดไป

4.4 กฎการคำนวณ release_at

```
// app/Listeners/UpdateOrderEarningTable.php :: calculateNextReleaseAt()  
$paidAt  
->setTimezone('Asia/Bangkok')  
->startOfDay()  
->addDay()  
->setTime(8, 0, 0);
```

= 08:00 ของวันถัดไป หลังจากเวลาที่ลูกค้าชำระเงินเสร็จ

5. Operation Procedures (วิธีปฏิบัติ)

5.1 ตรวจสอบสถานะ scheduler

```
ssh ssm.toko.service '
  echo "=== Cron file ==="; sudo cat /etc/cron.d/laravel-scheduler
  echo; echo "=== Last 10 scheduler runs ==="; sudo tail -10 /var/log/laravel-scheduler.log
  echo; echo "=== Earning release log ==="; sudo tail -20 /var/app/current/storage/logs/earning-r
  echo; echo "=== Wallet jobs log ==="; sudo tail -20 /var/app/current/storage/logs/wallet-jo
'
```

ผลลัพธ์ที่ถูกต้อง

- บรรทัด cron ขึ้นต้นด้วย * * * * * root bash -lc ...
- earning-release log แสดง Released N pending earning movement(s). หรือไม่มี exception
- ไม่พบ Permission denied หรือ SQLSTATE[HY000] [2002] Connection refused

5.2 รันโยกเงินด้วยมือ (catch-up หลัง cron พัง)

```
ssh ssm.toko.service "sudo bash -c '
  set -a; . /opt/elasticbeanstalk/deployment/env; set +a
  cd /var/app/current && sudo -E -u webapp php artisan earning:release-pending
'"
```

ผลลัพธ์: Released N pending earning movement(s).

5.3 ตรวจสอบรายได้ร้านใน DB

-- ดูยอดรวม

```
SELECT market_id, total_earning, next_day_earning, next_release_at
FROM earnings
WHERE market_id = ?;
```

-- ดูรายการรายได้ที่ยังไม่ปล่อย

```
SELECT id, order_id, market_earning, release_at, release_status
FROM earning_logs
WHERE market_id = ? AND release_status = 'pending'
ORDER BY release_at ASC;
```

5.4 ดู log error_log() ของ PHP

```
ssh ssm.toko.service 'sudo tail -100 /var/log/php-fpm/www-error.log'
```

6. Troubleshooting

6.1 รายได้ค้างไม่ปล่อยหลายวัน

อาการ: ยอดในแอปฝั่งร้านโชว์ “รายได้รอปล่อย” สะสมเรื่อย ๆ ไม่โยกเข้า “ถอนได้”

ขั้นตอนตรวจสอบ

1. SSH ดู `/var/log/laravel-scheduler.log` — ถ้าเจอ `Permission denied` ของ env file -> cron รับเป็น webapp ผิด user ดู §6.2
2. ดู `storage/logs/earning-release.log` — ถ้าเจอ `SQLSTATE[HY000] [2002] Connection refused` -> DB env ไม่ถูก source ดู §6.2
3. ถ้า cron ปกติแต่ยอดยังค้าง -> query `earning_logs` ดูว่า `release_at > NOW()` (ยังไม่ถึงเวลา) หรือเปล่า

แก้ปัญหา

- รับ catch-up ตาม §5.2
- ถ้า cron พัง -> ติดตั้ง cron ใหม่ตาม §6.2

6.2 Cron ไม่อ่าน DB credentials

สาเหตุ: `/opt/elasticbeanstalk/deployment/env` มี permission `0400` root:root ผู้ใช้ webapp อ่านไม่ได้

Hot-fix บน prod

```
ssh ssm.toko.service "sudo tee /etc/cron.d/laravel-scheduler > /dev/null <<'EOF'
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
* * * * * root bash -lc 'set -a; . /opt/elasticbeanstalk/deployment/env; set +a; cd /var/app/current
EOF
sudo chmod 0644 /etc/cron.d/laravel-scheduler
sudo systemctl reload crond"
```

Permanent fix: แก้ `.platform/hooks/postdeploy/03_install_scheduler_cron.sh` ให้ install entry นี้ใน (commit ไว้แล้วใน repo)

6.3 ยอดถอน / next_day ไม่ตรง

หากเจอเคสยอดเพี้ยน

— ตรวจสอบ pending ที่เหลือ vs next_day_earning

```
SELECT
  e.market_id,
  e.next_day_earning,
  e.next_day_orders,
  COALESCE(SUM(l.market_earning), 0) AS sum_pending,
  COUNT(l.id) AS cnt_pending
FROM earnings e
LEFT JOIN earning_logs l
  ON l.market_id = e.market_id AND l.release_status = 'pending'
WHERE e.market_id = ?
GROUP BY e.market_id;
```

ถ้า `next_day_earning != sum_pending` แสดงว่ามี drift -> ใช้คำสั่ง `BackfillReleasedEarningLogs` หรือเปิด ticket ให้ทีม backend ตรวจสอบ

7. Deploy Checklist

- ☐ `git push codecommit main` (CodePipeline จะ deploy ให้อีก)
- ☐ ตรวจสอบ Beanstalk environment กลับเป็น **Health: Ok**
- ☐ รับ \$5.1 ๑ cron + log หลัง deploy 2-3 นาที
- ☐ ถ้าเปลี่ยน schema: SSH รับ `php artisan migrate --force` (manual)
- ☐ ทดสอบสร้างออเดอร์ -> ตรวจสอบ `earning_logs` มี row ใหม่ status **pending**
- ☐ วันถัดไปหลัง 08:00 ตรวจสอบว่า status เปลี่ยนเป็น **released**

8. Reference

- Source code:
 - app/Console/Kernel.php — schedule definition
 - app/Console/Commands/ReleasePendingEarnings.php — earning release worker
 - app/Console/Commands/ProcessWalletJobs.php — wallet queue worker
 - app/Console/Commands/AutoCancelExpiredTableBookings.php — booking auto-cancel
 - app/Listeners/UpdateOrderEarningTable.php — สร้าง earning_logs ตอน payment.paid
 - .platform/hooks/postdeploy/03_install_scheduler_cron.sh — cron installer
- เอกสารอื่น:
 - TOKO_Shop_Manual.md — คู่มือใช้งานสำหรับร้านค้า

ewpage

9. Notification Resend & Auto-Reassign

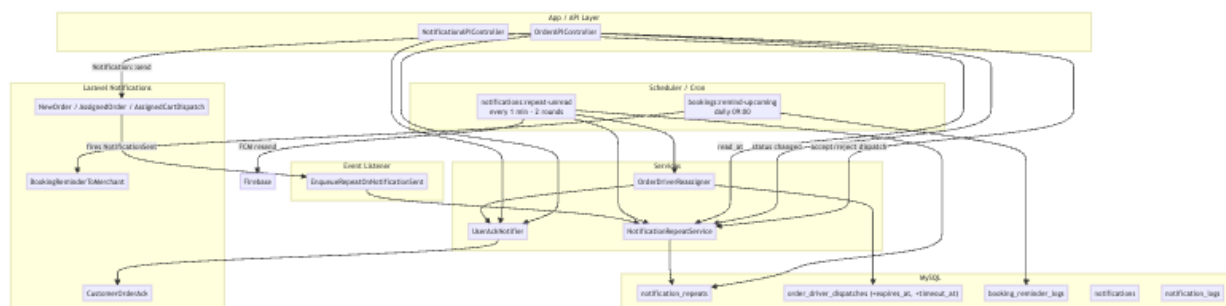
เพิ่มเข้ามาใน release 2026-05-07 — ระบบส่งซ้ำการแจ้งเตือน, Auto-reassign คนขับ, Acknowledgement กลับลูกค้า, Booking Reminder

1. ภาพรวมระบบ (What changed)

ก่อนหน้านี้ TOKO ยัง notification “ครั้งเดียว” ถ้าผู้รับไม่เห็น/ไม่กดอ่าน -> งานหลุด ระบบใหม่นี้แก้ปัญหาโดย:

1. **Resend ทุก** 30 วินาที จนกว่าผู้รับจะอ่าน/ตอบรับ หรือครบ max attempts
2. **Auto-reassign คนขับ** เมื่อ dispatch หมดเวลา (180 วินาที)
3. **แจ้ง rider** ถ้า งานถูกย้ายไปคนอื่นแล้ว
4. **Acknowledge ลูกค้า** ทุก milestone: ร้านเห็นแล้ว / ร้านรับแล้ว / ไสเดอร์รับแล้ว / กำลังหาไสเดอร์ใหม่ / ไม่มีไสเดอร์ว่าง
5. **เตือน merchant** ก่อนถึงวันจอง 1 วัน + เข้าวันจอง (Hotel/Service) ผ่าน Notification + Email

2. สถาปัตยกรรม (Architecture)



3. ตารางฐานข้อมูลที่เพิ่ม

3.1 notification_repeats (คิวส่งซ้ำ)

Column	Type	Note
id	bigInt	PK
notification_id	uuid	FK ไปที่ notifications.id (สำหรับ stop เมื่อ user อ่าน)

Column	Type	Note
user_id	bigInt	ผู้รับ
market_id	bigInt nullable	ร้านที่เกี่ยวข้อง
kind	varchar(50)	เช่น new_order_dinein, assigned_order_rider, hotel_booking
order_id	bigInt nullable	
dispatch_id	bigInt nullable	FK order_driver_dispatches
bookable_type / bookable_id	string / bigInt	สำหรับ booking (hotel/service)
title / body / data(json)		payload สำหรับ resend
status	varchar	active / stopped / expired
stop_reason	string	เหตุผลที่หยุด เช่น read, accepted, timeout, max_attempts
send_count / max_attempts	int	นับรอบส่ง
interval_seconds	int	default 30
last_sent_at / next_send_at / expires_at	datetime	

3.2 order_driver_dispatches (เพิ่มคอลัมน์)

Column	Type	Note
expires_at	datetime nullable	เช็ท now() +180s ตอน dispatch ใ้ไม่
timeout_at	datetime nullable	เช็ทเมื่อ reassigner ตัดทิ้ง
reassigned_from_dispatch_id	bigInt nullable	trace ลูกโซ่

3.3 booking_reminder_logs (กันส่งซ้ำ)

Column	Type	Note
bookable_type	string	MarketTableBooking
bookable_id	bigInt	
kind	enum-like	day_before / day_of
user_id	bigInt	merchant ที่รับ
sent_at	datetime	
UNIQUE	(bookable_type, bookable_id, kind, user_id)	

4. การตั้งค่าก่อน Deploy

4.1 Cron / Scheduler

ตรวจให้แน่ใจว่ามี cron entry บนเครื่อง production:

```
* * * * * cd /var/www/toko-server-v2 && php artisan schedule:run >> /dev/null 2>&1
```

ระบบเดิมก็ใช้ scheduler อยู่แล้ว (wallet:process-jobs, earning:release-pending) — ถ้ามีอยู่แล้วไม่ต้องเพิ่ม

4.2 ENV / Firebase

ใช้ **FirebaseService** เดิม — ไม่ ต้อง ตั้ง ค่า ใหม่ - ตรวจ **firebase-credentials.json** มีอยู่และอ่านได้ - กรณี environment เก่ายังใช้ legacy server key ก็ยังใช้ได้

4.3 Email สำหรับ Booking Reminder

- ใช้ mail driver เดิมที่ตั้งใน .env (MAIL_*)
- merchant user ต้องมี email ใน DB ถึงจะส่ง email (ไม่มี ก็ส่งเฉพาะ FCM + database)

4.4 Flutter Channel ID (SuperApp)

ฝั่ง toko_superapp_5 ต้องมี notification channel: - toko_customer_channel_v1 — สำหรับ Acknowledgement กลับลูกค้า

ถ้ายังไม่ให้เพิ่มที่ AndroidManifest / native code (เหมือนที่มี toko_merchant_channel_v2 อยู่แล้ว)

5. ขั้นตอน Deploy (Runbook)

1) Pull code

```
cd /var/www/toko-server-v2
git pull
```

2) ติดตั้ง dependencies (ไม่มี package ใหม่ แต่กันเหนียว)

```
composer install --no-dev --optimize-autoloader
```

3) Migrate (□ □ □ □ 3 ตาราง)

```
php artisan migrate --force
```

4) Cache เคลียร์ (กับ config/route cache เท่า)

```
php artisan config:clear
php artisan route:clear
php artisan cache:clear
```

5) (□ □ □ □ □ supervisor / queue worker / scheduler) restart

```
sudo supervisorctl restart all
```

ตรวจสอบ Deploy

A) ตรวจสอบ migrations ขึ้นคน

```
php artisan migrate:status | grep -E "notification_repeats|booking_reminder|order_driver_dispatches"
```

B) ตรวจสอบ command ขึ้น

```
php artisan list | grep -E "notifications:repeat-unread|bookings:remind-upcoming"
```

C) Smoke test ครั้งเดียว

```
php artisan notifications:repeat-unread --once --limit=10
php artisan bookings:remind-upcoming --dry-run
```

ผลที่คาดหวัง:

```
Firestore initialized with service account file
[2026-05-07 22:16:04] resend=0 reassign=0
```

6. การทำงานของ Background Jobs

6.1 notifications:repeat-unread (every minute)

สอบ	เวลา (วินาที)	งานที่ทำ
1	0	resend FCM + sweep dispatch timeout
sleep	0-30	50
2	30	resend FCM + sweep dispatch timeout

ผลคือ resend ได้ “ทุก 30 วินาที” ทั้งที่ Laravel scheduler ส่งรับขึ้นค่า everyMinute

6.2 bookings: remind-upcoming (daily 09:00)

- หา MarketTableBooking ที่ booking_date = วันพรุ่งนี้ -> ส่ง day_before
- หา MarketTableBooking ที่ booking_date = วันนี้ -> ส่ง day_of
- ส่งให้ทุก user ที่ผูกกับ market นั้น (user_markets)
- กันส่งซ้ำด้วย unique index บน booking_reminder_logs

7. Per-kind Defaults (Resend Policy)

kind	interval	max_attempts	รวมเวลา
new_order_dinein	30s	20	~10 นาที
new_order_pickup	30s	20	~10 นาที
new_order_delivery_merchant	30s	20	~10 นาที
assigned_order_rider	30s	6	~3 นาที (= dispatch timeout 180s)
hotel_booking	30s	30	~15 นาที
service_booking	30s	30	~15 นาที
field3_order	30s	20	~10 นาที
field45_order	30s	20	~10 นาที

ปรับได้ที่ App\Services\NotificationRepeatService::kindDefaults()

8. Stop Conditions (เมื่อไรหยุดส่งซ้ำ)

สถานการณ์	ตำแหน่งใน code	reason
User อ่าน notification	NotificationAPIController@update (read_at)	read
ร้านรับ order (เปลี่ยน status)	OrderAPIController@update	accepted_by_merchant
ไรเดอร์กด accept	OrderAPIController@acceptDriverDispatch	dispatch_accepted
ไรเดอร์กด reject	OrderAPIController@rejectDriverDispatch	dispatch_rejected
Dispatch หมดเวลา 180s	OrderDriverReassigner::reassignByTimeout	ByTimeout
ครบ max_attempts	consumer command	max_attempts
expires_at หมด	consumer command	expired
Order ปิด/ยกเลิก	stopForOrder()	accepted_by_merchant (ขยายต่อได้)

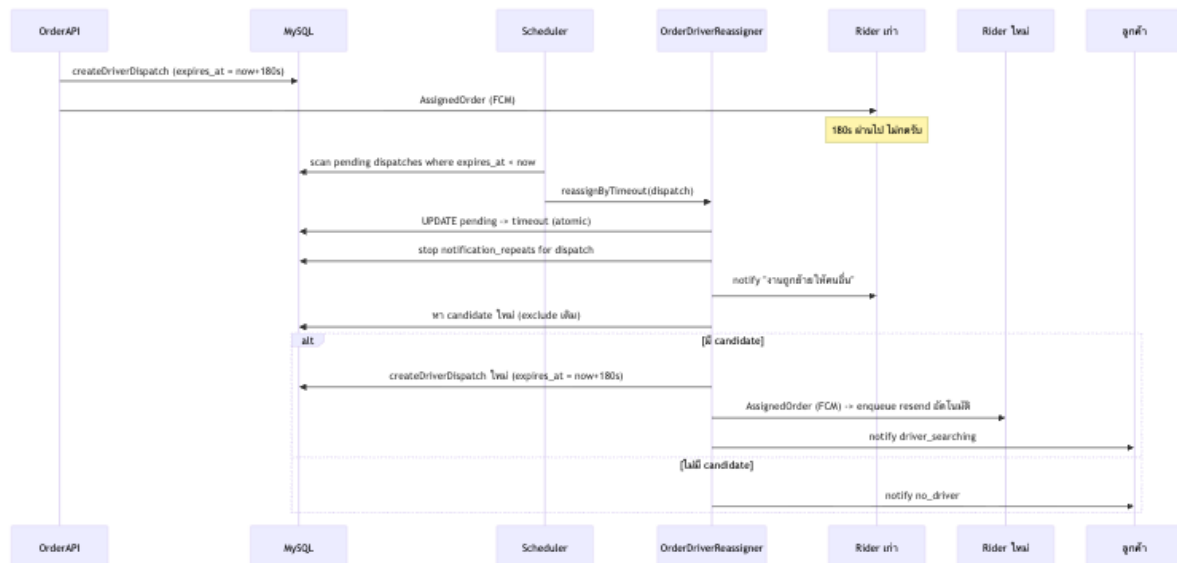
9. Acknowledgement Notifications (ที่ลูกค้าจะได้รับเพิ่ม)

ทุก event ส่งเป็น database + fcm channel ผ่าน CustomerOrderAck:

event	trigger	ตัวอย่างข้อความ
merchant_seen	ร้านอ่าน noti	“ร้านค้าเห็นออเดอร์ #123 ของคุณแล้ว”
merchant_accepted	ร้านเปลี่ยน order_status	“ร้านค้ารับออเดอร์ #123 และเริ่มเตรียมแล้ว”
driver_seen	ไรเดอร์อ่าน noti	“ไรเดอร์เห็นออเดอร์ #123 ของคุณแล้ว”
driver_accepted	ไรเดอร์ accept dispatch	“ไรเดอร์ {ชื่อ} กำลังจะไปรับออเดอร์ #123”
driver_searching	reject / reassign สำเร็จ	“ระบบกำลังหาไรเดอร์ใหม่ให้กับออเดอร์ #123”
no_driver	reject / reassign ไม่เจอ candidate	“ขณะนี้ยังไม่มีไรเดอร์ว่างสำหรับออเดอร์ #123 ระบบจะลองอีกครั้ง”
reassigned_to_other	rider เก่า ตอน timeout	“ออเดอร์ #123 ถูกมอบหมายให้ไรเดอร์ท่านอื่น”

ฝั่ง SuperApp ควรจัดการ event แต่ละชนิดเพื่อแสดง UI ให้เหมาะสม (toast / badge / sound)

10. Auto-Reassign Flow (Driver Dispatch Timeout)



11. Monitoring / Troubleshooting

Logs

- storage/logs/notifications-repeat.log — output ของ consumer command
- storage/logs/booking-reminders.log — output ของ booking command
- storage/logs/laravel.log — error / warning ที่ทั่วไป

Quick checks (SQL)

-- คิว resend ที่ยัง active วันนี้

```
SELECT kind, status, COUNT(*) FROM notification_repeats
WHERE created_at >= CURDATE()
GROUP BY kind, status;
```

-- Dispatch ที่กำลังจะ timeout ใน 1 นาทีข้างหน้า

```
SELECT id, order_id, driver_id, expires_at FROM order_driver_dispatches
WHERE status='pending' AND expires_at BETWEEN NOW() AND NOW()+INTERVAL 60 SECOND;
```

-- Booking reminders ที่ส่งวันนี้

```
SELECT kind, COUNT(*) FROM booking_reminder_logs
WHERE DATE(sent_at) = CURDATE() GROUP BY kind;
```

-- เช็ก resend logs

```
SELECT notification_type, status, method, COUNT(*) FROM notification_logs
WHERE method LIKE 'repeat:%' AND created_at >= NOW()-INTERVAL 1 HOUR
GROUP BY notification_type, status, method;
```

อาการ / สาเหตุ

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
ไม่ resend เลย	scheduler ไม่ทำงาน	ตรวจ cron entry, log schedule:run
Resend แล้วแต่ FCM ไม่ถึง	token ผิด / channel id ไม่ตรง	ตรวจ user_device_tokens.is_active, channel id ผิด app
Reassign แล้วได้ rider เดิม	exclude list ผิด	ตรวจ order_driver_dispatches.driver_id ของ order/cart นั้น
Booking reminder ส่งซ้ำ 2 ครั้ง	unique index ไม่ถูกสร้าง	SHOW INDEX FROM booking_reminder_logs
Customer ไม่เห็น ack	channel toko_customer_channel_v1 ผิด app ไม่มี	สร้าง channel ผิด Flutter

12. Rollback Plan

ถ้ากลับ deploy พบปัญหา:

```
## 1) ปิด scheduler entries (ลบ/comment สอง command ใน app/Console/Kernel.php)
## notifications:repeat-unread, bookings:remind-upcoming
## หรือใช้ feature flag (ถ้าเพิ่ม)
```

2) Revert code

```
git revert <commit-hash>
git push
```

3) Optional: drop ตารางใหม่ (ระวัง data loss สำหรับ logs)

```
php artisan migrate:rollback --step=3
```

ตารางที่เพิ่มใหม่ทั้ง 3 เป็น append-only (ไม่ทับโครงสร้างเดิม) ดังนั้น rollback code อย่างเดียวก็ปลอดภัย

13. ผลกระทบต่อระบบเดิม (Side effects checklist)

ส่วน	ผลกระทบ	ระดับ
<code>NewOrder::via()</code> เพิ่ม 'fcm'	ก่อนหน้านี้ส่งแค่ database — ตอนนี้อย่าง FCM ครั้งแรกด้วย	กลาง — ต้องตรวจว่า merchant device token พร้อม
<code>createDriverDispatch</code> เพิ่ม <code>expires_at</code>	dispatch ใหม่จะ auto-timeout 180s	สูง — เปลี่ยนพฤติกรรม dispatch
<code>OrderApiController@update</code> ส่ง <code>ack</code> ลูกค้า	ลูกค้าจะได้ noti เพิ่ม	ต่ำ — ดี
Notification ใหม่	volume noti เพิ่ม	ต่ำ
<code>CustomerOrderAck</code>		
Booking reminder daily 09:00	merchant อาจได้ email/noti จำนวนหนึ่ง	ต่ำ

14. ไฟล์ที่ต้องเช็ก Reviewer ดู

Backend (toko- server- v2) - `app/ Services/ NotificationRepeatService.php` - `app/ Services/ OrderDriverReassigner.php` - `app/Services/UserAckNotifier.php` - `app/Listeners/EnqueueRepeatOnNotification` - `app/Console/Commands/RepeatUnreadNotifications.php` - `app/Console/Commands/SendBookingReminders.php` - `app/Notifications/Customer/CustomerOrderAck.php` - `app/Notifications/BookingReminderToMerchant.php` - `app/Notifications/NewOrder.php` (เพิ่ม fcm channel) - `app/Http/Controllers/API/OrderApiController.php` (stop wiring) - `app/Http/Controllers/API/NotificationApiController.php` (stop wiring) - `app/Console/Kernel.php` (schedule) - `database/migrations/2026_05_07_120000_*.php` (3 ไฟล์)

Frontend (ตามมาก็หลัก) - `toko_superapp_5`: handle event `merchant_seen` / `merchant_accepted` / `driver_*` ใน `NotificationService` - `toko_superapp_5`: เพิ่ม Android channel `toko_customer_channel_v1`

15. Sign-off Checklist (ก่อน Production)

- ☐ `php artisan migrate:status` ขึ้นครบ 3 รายการใหม่
- ☐ `php artisan list` พบ 2 commands ใหม่
- ☐ Cron * * * * * `php artisan schedule:run` ทำงาน (ตรวจ `schedule:list` หรือ log)
- ☐ Smoke test resend command ผ่าน (`--once` ได้ผลลัพธ์ `resend=0 reassign=0`)
- ☐ Smoke test booking command ผ่าน (`--dry-run` แสดง list)
- ☐ ส่ง test order: merchant ได้ noti, ปล่อยทิ้ง 30s ได้รอบลอง
- ☐ ส่ง test order ให้ rider แล้วไม่กด — ภายใน ~3 นาทีต้อง reassign
- ☐ ลูกค้าได้ ack noti ครบ flow
- ☐ Channel `toko_customer_channel_v1` ผัง SuperApp พร้อม